

jordanvalnet/code-dev-intel.ts

# code-dev-intel : une brique d'intelligence de code pour agents IA, bien fabriquée, portée par une seule personne sous AGPL — à adopter en interne, pas à intégrer

Self-hosted AI code intelligence for TypeScript — symbol resolution, structural search, duplicate detection, and dependency graphs via MCP server.

1

BUS FACTOR  
sur 12 semaines

11

COMMITTS  
sur 12 semaines

14

DÉPENDANCES  
5 en production

AGPL-3.0

LICENCE

<https://github.com/jordanvalnet/code-dev-intel.ts> Licence AGPL-3.0 Onboard · profil cto

# Que fait ce projet et quelle est son architecture ?

Un service local de navigation de code pour agents IA : quatre services autour d'un seul exécutable, aucune dépendance à un indexeur distant.

|                              |                                              |
|------------------------------|----------------------------------------------|
| <code>.github/</code>        | workflows GitHub Actions                     |
| <code>__tests__/</code>      | tests unitaires vitest                       |
| <code>docker/</code>         | docker-compose.yml et profils                |
| <code>docs/</code>           | documentation, dont docs/ai                  |
| <code>perf/</code>           | benchmarks de performance                    |
| <code>schemas/</code>        | schémas livrés avec le paquet                |
| <code>scripts/</code>        | bootstrap, smoke tests, benchmark            |
| <code>security/</code>       | ressources sécurité                          |
| <code>services/</code>       | serveur MCP, indexeur, gouvernance, sécurité |
| <code>.dockerignore</code>   | exclusions du build Docker                   |
| <code>.gitignore</code>      | fichiers ignorés par git                     |
| <code>.npmignore</code>      | exclusions du paquet npm                     |
| <code>CHANGELOG.md</code>    | journal des versions                         |
| <code>CONTRIBUTING.md</code> | guide de contribution                        |
| <code>LICENSE</code>         | licence AGPL-3.0-only                        |
| <code>README.md</code>       | présentation et documentation principale     |

... et 6 autres entrées à la racine du dépôt

`code-dev-intel.ts` est un serveur MCP auto-hébergé qui répond aux questions de navigation de code d'un agent IA sur un dépôt TypeScript : définitions, références, implémentations, hiérarchie d'appels, graphe d'imports, rayon d'impact d'un changement, recherche structurelle et détection de duplication.

Trois surfaces d'accès pour un seul exécutable : MCP sur stdio, MCP en JSON-RPC sur `POST /mcp`, et des points HTTP `/tools/*` avec `/health` et `/tools/describe`.

L'architecture est un monodépôt de services : `services/` porte le serveur MCP, l'indexeur, la gouvernance et la sécurité ; le reste de la racine est de l'outillage — `scripts/`, `perf/`, `schemas/`, `docker/`, `__tests__/`.

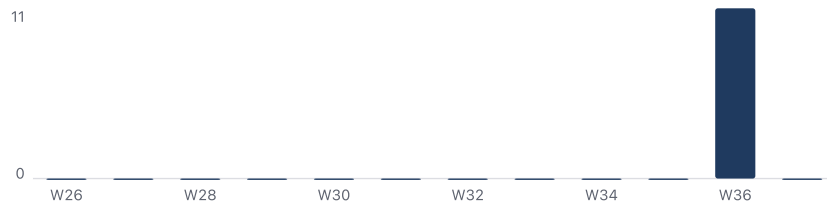
Point d'entrée unique côté produit : `services/code-intel-mcp/src/server.ts`, à la fois `main` et `bin` du paquet une fois compilé dans `dist/`.

La valeur technique n'est pas dans les outils exposés mais dans leur résolution : le graphe d'imports passe par `ts.resolveModuleName` avec le `tsconfig.json` le plus proche du fichier importateur, et rend compte de ce qu'il n'a pas su suivre au lieu de le taire.

Distribution : paquet npm public installé en dépendance de développement, démarré par une commande unique conçue pour les scripts et la CI.

## Quelle est sa santé technique ?

Ingénierie de livraison sérieuse — matrice trois systèmes, smoke test du tarball — mais cinq mois d'écart entre la dernière version estampillée et le code du dépôt.



Le rythme est discontinu : 11 des commits relevés sur douze semaines tombent dans la seule semaine du 1er septembre 2026, les onze semaines précédentes sont à zéro ; dernier commit le 6 septembre 2026.

La chaîne de vérification, elle, est nettement au-dessus de ce qu'on attend d'un projet à un contributeur : quatre workflows, et `ci.yml` enchaîne lint, type-check et tests sur ubuntu, windows et macos en Node 20, 22 et 24, puis `release-smoke`, `indexer-smoke` et un scan de sécurité, à chaque push et chaque pull request.

`release-smoke` va jusqu'au bout de la chaîne : il empaquette le tarball qu'une publication produirait, l'installe dans un projet jetable sur les trois systèmes et pilote le serveur installé — c'est là que se voient une erreur de `files`, une dépendance optionnelle absente ou un bug de chemin Windows, invisibles depuis l'arbre source.

Tests : `vitest`, quatre ensembles sous `__tests__/unit` (serveur MCP, indexeur, gouvernance, sécurité), lancés par `pnpm test`.

Le trou est ailleurs : une seule version estampillée, v0.1.6 du 26 mars 2026, alors que le manifeste du dépôt est en 0.5.0 et que le journal des versions décrit un 0.5.0 daté du 4 septembre 2026.

Cinq mois de travail sans étiquette publique : la page des versions ne dit plus où en est le produit, et un adoptant ne sait pas ce qu'il installe<sup>1</sup>.

## Qui porte le projet et quel est le bus factor ?

Une personne, un compte personnel, aucun responsable de secours désigné : le premier risque de ce dépôt est humain, pas technique.

jordanvalnet  12

Bus factor 1 : un seul compte, `jordanvalnet` , porte la totalité des commits relevés — 12 à son compteur sur la période.

Le dépôt vit sur un compte personnel et non sur une organisation, et aucun fichier `CODEOWNERS` ne désigne de responsable de secours.

Le process, lui, est écrit et exigeant : branches courtes `task/T-00X-short-name` , une tâche par pull request, définition de fini en cinq points dont `pnpm lint` , `pnpm type-check` et `pnpm test` , et un journal de mémoire partagée à compléter à chaque tâche.

Ce process est calibré pour des agents IA travaillant sur `docs/ai/` , pas pour une communauté : aucune contribution extérieure relevée, aucune issue étiquetée pour un premier contributeur.

Parade, niveau élevé : obtenir un second mainteneur avec droit de publication avant tout usage en production, sinon traiter le fork interne comme le plan de continuité — la licence l'autorise expressément.

## Quelles dépendances et quelle dette ?

La dette n'est pas dans les paquets — cinq dépendances d'exécution, verrou, versions vulnérables épinglées — elle est dans le `engines.node >=18` que rien ne vérifie.

14 dépendances déclarées dans `package.json` : 5 d'exécution, 9 de développement.

| DÉPENDANCE D'EXÉCUTION       | VERSION              | CE QU'ELLE PORTE                                                |
|------------------------------|----------------------|-----------------------------------------------------------------|
| <code>typescript</code>      | <code>^6.0.3</code>  | résolution de modules et vérificateur de types, cœur du produit |
| <code>@ast-grep/cli</code>   | <code>^0.45.3</code> | recherche structurelle, binaire natif par plateforme            |
| <code>@vscode/ripgrep</code> | <code>^1.18.0</code> | recherche texte, binaire natif embarqué                         |
| <code>zod</code>             | <code>^4.5.4</code>  | validation des entrées d'outils                                 |
| <code>picomatch</code>       | <code>^4.0.7</code>  | motifs d'exclusion de fichiers                                  |

Surface d'attaque contenue : cinq paquets d'exécution seulement, verrou `pnpm-lock.yaml` , et dix plages de versions vulnérables épinglées à la main via `pnpm.overrides` .

Contrepartie assumée : deux binaires natifs et sept dépendances optionnelles par plateforme — c'est ce qui rend la matrice trois systèmes obligatoire et non confortable.

Dette n° 1, déclarative : `engines.node` annonce `>=18.0.0` alors que l'outillage de développement exige Node 22 ou plus (vite5, `--experimental-strip-types` ) ; seul le paquet compilé est vérifié sur Node 20, et Node 18 n'est testé nulle part.

Dette n° 2, documentaire : aucun repère d'agent à la racine — 0 sur 7, ni `AGENTS.md` , ni `CLAUDE.md` , ni `.mcp.json` — alors que le produit s'adresse aux agents ; la documentation existe mais est rangée sous `docs/ai/` .

Dette de code, en revanche, quasi nulle : la recherche de marqueurs `TODD` ne remonte que deux occurrences, toutes deux dans des textes de description d'outils, aucun chantier laissé en commentaire.

## Quels risques ?

Les deux risques élevés sont non techniques — la licence et la personne unique — et tous deux se traitent par contrat, pas par correctif.

- **license** AGPL-3.0 : copyleft, contraignant pour un usage commercial.
- **security** Pas de SECURITY.md : aucun canal déclaré pour signaler une faille.
- **bus\_factor** Un seul contributeur porte la moitié des commits sur 12 semaines.
- **ci** Quatre workflows dans .github/workflows ; ci.yml enchaîne lint, type-check et tests sur ubuntu, windows et macos (node 20, 22, 24), puis release-smoke, indexer-smoke et scan sécurité, à cha...
- **deps** 5 dépendances d'exécution et 9 de développement dans package.json, verrou pnpm-lock.yaml, versions vulnérables épinglées via pnpm.overrides ; engines annonce node >=18 mais ci.yml précise q...

Cinq risques relevés : deux élevés, un moyen, deux faibles.

| RISQUE               | NIVEAU | PARADE                                                                                                                                                                        |
|----------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Licence              | ÉLEVÉ  | Usage interne sans modification : sans effet. Toute intégration à un produit vendu ou exposé en réseau : avis juridique, ou licence commerciale négociée avec l'auteur unique |
| Bus factor           | ÉLEVÉ  | Second mainteneur avec droit de publication, ou fork interne épinglé sur une version audité                                                                                   |
| Sécurité             | MOYEN  | Contact direct de l'auteur documenté côté adoptant, et scan <code>pnpm security:scan</code> rejoué chez vous                                                                  |
| Intégration continue | FAIBLE | Rien à faire : vérifier seulement que la matrice reste à trois systèmes lors des montées de version                                                                           |
| Dépendances          | FAIBLE | <code>pnpm install --frozen-lockfile</code> , relecture des <code>pnpm.overrides</code> à chaque montée de version                                                            |

Le canal de signalement manque vraiment : aucune politique de sécurité n'est publiée, donc aucune adresse où envoyer une faille, alors même qu'un dossier `security/` existe à la racine et qu'un scan OpenGrep tourne en intégration continue.

Donnée non relevée : les alertes de vulnérabilité des dépendances, hors de portée des moyens d'accès utilisés ici, à vérifier sur la page sécurité du dépôt<sup>2</sup>.

## Adopter, contribuer ou forker ?

Adopter en interne sans parier sur la communauté : elle n'existe pas encore, et le fork reste ouvert si l'auteur s'arrête.

La feuille de route publique est vide ou presque : aucune pull request ouverte, une seule demande ouverte, « Anti-Amnesia & Workflow Hardening », sans commentaire, et aucun thème qui se dégage.

Le seul mouvement récent est interne : la pull request « ci: three-OS quality matrix, multi-OS release smoke, per-OS perf budget », fusionnée le 6 septembre 2026 — de la mise en qualité, pas de la fonctionnalité.

Donnée non relevée : les jalons de planification, non exposés par les moyens d'accès utilisés ici, à vérifier sur la page des jalons du dépôt<sup>3</sup>.

- **Adopter** : oui, en interne. Installation en dépendance de développement, démarrage à la demande en CI ou en hook, rien ne quitte vos machines.
- **Contribuer** : possible, à faible rendement. La file est vide, il n'y a personne à convaincre, et le process de contribution est écrit pour des agents suivant un backlog interne.
- **Forker** : c'est la sortie de secours contre le bus factor 1, et la licence vous y autorise ; le coût est de reprendre un monodépôt TypeScript de quatre services.

## Décision

**Adopter en pilote interne, ne rien intégrer à un produit.** Le gain annoncé est mesuré et l'engagement est réversible : une dépendance de développement, un serveur local, aucune donnée qui sort — 11 agents sur 11 l'ont choisi spontanément, pour 13 à 34 % de jetons en moins sur les tâches difficiles.

1. **Épingler une version connue et l'auditer.** Le dépôt est en 0.5.0, la dernière version estampillée est v0.1.6 du 26 mars 2026 : installez depuis un commit choisi, pas depuis une étiquette qui a cinq mois de retard.
2. **Bloquer toute intégration produit avant avis juridique.** AGPL-3.0, risque élevé : l'usage interne est libre, l'exposition en réseau d'un dérivé oblige à publier votre propre code.
3. **Traiter le bus factor 1 comme une condition, pas comme une réserve.** Un contributeur, aucun responsable de secours désigné : fork interne tenu à jour, ou second mainteneur, avant tout usage en production.
4. **Mesurer avant de généraliser.** Un mois, une équipe volontaire, la consommation de jetons avant et après : le seul chiffre disponible aujourd'hui est celui de l'auteur, sur sa propre base de code<sup>4</sup>.



Sources

1. <https://github.com/jordanvalnet/code-dev-intel.ts/releases>
2. <https://github.com/jordanvalnet/code-dev-intel.ts/security>
3. <https://github.com/jordanvalnet/code-dev-intel.ts/milestones>
4. <https://github.com/jordanvalnet/code-dev-intel.ts/blob/main/docs/benchmarks/2026-06-07-agent-token-economy.md>